KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

WriteWise AI: A Retrieval-Augmented Generation (RAG)-Based Email and Formal Letter Writing Assistant

Project ID: PRJ-019

Team ID: TEAM TITANS

Team GitHub Repository: https://github.com/akashvarma1338/ICL-GEN-AI

Student Name	Registration Number	Individual GitHub Link
Vanapala Akash Varma	99230041018	https://github.com/akashvarma1338/ICL-GEN-AI
Karli Tejasree	99230040872	https://github.com/tejasreekarli/GEN-AI-MASTERY-COURSE
Devarnnagari Deekshitha	99230040924	https://github.com/devarannagarideekshitha/Gen-AI-_Assignment
Nelluri Vinay	99230040941	https://github.com/nellurivinay1902
P Gajapathi	99230040720	https://github.com/Gajachowdary/ICL-project

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1
2. Problem Statement & Objectives ..... Section 2
3. System Architecture (RAG Pipeline) ..... Section 3
4. Implementation Details & Tech Stack ..... Section 4
5. Results and Testing ..... Section 5
6. Conclusion & Future Scope ..... Section 6

1 Introduction

In today's digital environment, writing professional emails and formal letters is an essential requirement for students, employees, and organizations. However, creating well-structured, grammatically correct, and contextually appropriate communication can be challenging and time-consuming.

WriteWise AI is an AI-powered writing assistant developed using Retrieval-Augmented Generation (RAG) technology. The application helps users generate professional emails and formal letters based on their requirements and uploaded reference documents.

The system uses a Retrieval-Augmented Generation approach to retrieve relevant information from a knowledge base before generating content. It combines semantic search with Large Language Models (LLMs) to produce accurate, relevant, and professionally structured drafts.

The application supports multiple email categories and formal letter types. Users can customize the recipient, sender, subject, purpose, details, tone, length, and language. It also provides editing, refinement, document retrieval inspection, and draft history features.

The primary goal of WriteWise AI is to simplify professional communication by providing an intelligent, user-friendly, and context-aware writing assistant.

2 Problem Statement & Objectives

Problem Statement

Writing professional emails and formal letters requires proper language, grammar, formatting, and communication etiquette. Students and professionals often face difficulties in preparing appropriate messages for different situations, such as job applications, leave requests, complaints, permissions, meeting requests, and academic communications.

Traditional writing tools mainly focus on grammar correction and basic text generation. They may not consider specific writing guidelines or reference documents provided by users. Additionally, general-purpose AI tools may generate inaccurate or irrelevant information when sufficient context is not available.

Therefore, there is a need for an intelligent writing assistant that can understand user requirements, retrieve relevant guidelines from uploaded documents, and generate personalized, well-structured, and contextually relevant emails and formal letters.

Objectives:

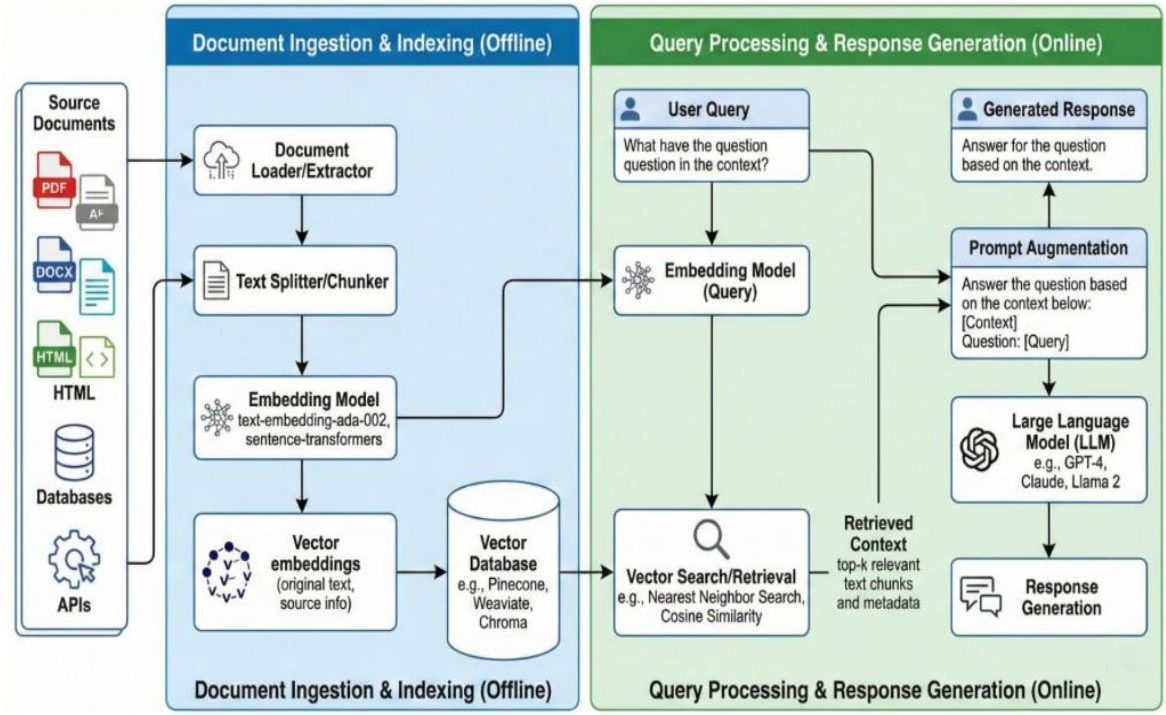
The main objectives of the WriteWise AI project are:

1. To develop an AI-powered application for generating professional emails and formal letters.
2. To implement Retrieval-Augmented Generation (RAG) for generating context-aware content using reference documents.
3. To support multiple categories of emails and formal letters.
4. To implement document upload and processing functionality for PDF and TXT files.
5. To use semantic search and vector databases for retrieving relevant information.
6. To improve the quality of generated content through grammar correction, refinement, and regeneration options.
7. To provide editable outputs and downloadable text files.
8. To maintain a local history of previously generated drafts.
9. To reduce inaccurate or fabricated information through context-based generation and prompt restrictions.

3 System Architecture (RAG Pipeline)

The architecture of WriteWise AI consists of two major phases:

- Data Ingestion Phase
- Query Processing and Generation Phase



Data Ingestion Phase

The Data Ingestion Phase is responsible for collecting, processing, and storing reference documents in the knowledge base. The system supports uploading multiple PDF and TXT files containing email-writing guidelines, formal letter formats, and other reference information.

The process involves the following steps:

- **1. Document Upload:**
  Users can upload multiple PDF and TXT documents through the Streamlit application.
- **2. Text Extraction:**
  The uploaded PDF documents are processed using the PyPDF library to extract text page by page. TXT files are read using UTF-8 encoding.
- **3. Text Chunking:**
  The extracted text is divided into smaller, meaningful chunks using Lang Chain's Recursive Character Text Splitter. The implemented configuration uses a chunk size of 500 characters and an overlap of 100 characters.
- **4. Embedding Generation:**
  Each text chunk is converted into a numerical vector representation using the Sentence Transformers model all-MiniLM-L6-v2.
- **5. Vector Database Storage:**
  The generated embeddings and associated metadata are stored in ChromaDB. The application uses a persistent ChromaDB client to retain the knowledge base locally.
- **6. Metadata Management:**
  Each chunk is associated with metadata such as source filename, chunk index, character count, and document ID. This information helps identify and inspect retrieved references

Query & Generation Phase

The Query Processing and Generation Phase generates personalized emails and formal letters based on user requirements and retrieved reference information.

The process consists of the following steps:

1. User Input:
The user selects an email or formal letter category and provides the necessary information, including sender, recipient, subject, purpose, details, tone, length, and language.

2. Query Construction:
The application prepares a search query based on the user's requirements and selected writing category.

3. Semantic Retrieval:
The query is compared with the stored document embeddings in ChromaDB. The system retrieves the most relevant text chunks based on semantic similarity, configurable top-K results, and similarity threshold.

4. Context Preparation:
The retrieved document chunks are formatted and added to the prompt along with the user's requirements.

5. Content Generation:
The prepared prompt is sent to the configured Large Language Model through the LLM service. Groq API with Llama 3.3 70B Versatile is the documented default configuration. The implementation also supports Gemini API dispatch.

6. Draft Generation:
The model generates a structured email with separate subject and body fields or a formal letter with the appropriate structure.

7. Draft Refinement:
Users can edit the generated content and apply refinement operations such as making it more formal, concise, expanding the content, improving grammar, or regenerating the draft.

8. Output and Storage:
The generated drafts can be downloaded as TXT files and are automatically saved in local writing history with timestamps and related information.

4 Implementation Details & Tech Stack

Component	Technology Used	Purpose
Programming Language	**Python**	**Core application logic**
Frontend Framework	**Streamlit**	**Interactive user interface**
Orchestration Framework	**LangChain**	**Connecting LLMs, document loaders, and vector stores**
Vector Database	**ChromaDB**	**Storing and retrieving document embeddings**
Embedding Model	**Sentence Transformers (all-MiniLM-L6-v2)**	**Converting text into vector representations**
Large Language Model	**Llama 3.3 70B Versatile**	**Generating context-aware emails and formal letters**
LLM API Provider	**Groq API**	**Connecting to the LLM for content generation**
Alternative LLM Provider	**Gemini API**	**Additional LLM integration**
PDF Processing	**PyPDF**	**Extracting text from PDF documents**
Text Splitting	**RecursiveCharacterTextSplitter**	**Dividing documents into smaller text chunks**
Data Storage	**JSON**	**Storing generated drafts and writing history**
Persistent Storage	**ChromaDB Persistent Client**	**Maintaining the local knowledge base**

4.1 Technology Description

1. Python:
Python is used as the primary programming language for implementing document processing, vector database operations, RAG workflows, and application logic.

2. Streamlit:
Streamlit is used to develop the interactive frontend. It provides different application pages for email writing, letter writing, knowledge base management, settings, and writing history.

3. LangChain:
LangChain is used for document loading, text splitting, and connecting different components involved in the Retrieval-Augmented Generation pipeline.

4. ChromaDB:
ChromaDB is used as a persistent vector database to store document embeddings and retrieve relevant text chunks through semantic similarity search.

5. Sentence Transformers:
The all-MiniLM-L6-v2 model converts text chunks into numerical vector representations, enabling semantic search.

6. Groq API:
Groq provides access to Large Language Models for generating context-aware emails and formal letters. The default configuration uses the Llama 3.3 70B Versatile model.

7. PyPDF:
PyPDF is used to extract text from uploaded PDF documents page by page.

8. Local JSON Storage:

JSON files are used to maintain writing history, including previously generated drafts and related information.

4.2 Sample Chunking Configuration

The application uses Lang Chain's Recursive Character Text Splitter to divide documents into smaller chunks while maintaining contextual continuity.
Configuration:

- Chunk Size: 500 characters
- Chunk Overlap: 100 characters
- Text Splitting Method: Recursive Character Text Splitter

4.3 Project Modules

The application is divided into the following major modules:

1. app.py:
Handles the Streamlit frontend, navigation, and interactive user interface.

2. config.py:
Maintains application configuration, default values, paths, and model constants.

3. document_processor.py:
Handles PDF and TXT document loading, text extraction, chunking, and metadata generation.

4. vector_store.py:
Manages ChromaDB operations, including storing embeddings, semantic querying, filtering, and document deletion.

5. llm_service.py:
Handles communication with Groq and Gemini APIs, LLM execution, and content refinement.

6. rag_pipeline.py:
Constructs search queries, retrieves relevant context, prepares prompts, and generates grounded content.

7. history_manager.py:
Manages locally stored writing history in JSON format.

4.4 Key Functionalities

- Email generation for 10 different categories.
- Formal letter generation for 10 different categories.
- Multiple PDF and TXT document uploads.
- Persistent knowledge base.
- Semantic similarity search.
- Retrieved reference inspection.
- Editable email and letter drafts.
- Content refinement and regeneration.
- TXT file download.
- Local writing history management.
- Individual document deletion and database reset.

5 Results and Testing

5.1 Results

The WriteWise AI application provides an interactive platform for generating professional emails and formal letters using Retrieval-Augmented Generation technology.

The application successfully integrates document processing, text chunking, embedding generation, semantic retrieval, and Large Language Model-based content generation.

The major results and functionalities include:

1. Successful uploading and processing of PDF and TXT reference documents.
2. Generation of document embeddings using the Sentence Transformers model.
3. Persistent storage and retrieval of document embeddings using ChromaDB.
4. Semantic retrieval of relevant reference information based on user queries.
5. Generation of structured and personalized emails and formal letters.
6. Availability of content refinement and editing functionalities.
7. Successful local storage and retrieval of previously generated drafts.
8. Downloading generated content in TXT format.

5.2 Testing

The application should be evaluated using different input scenarios to verify the correctness and reliability of its functionalities.

Testing should cover document uploading, text extraction, chunking, embedding generation, semantic retrieval, email generation, formal letter generation, content refinement, downloading, and writing history.

The results of each test case should be recorded after executing the corresponding operation.

5.3 Performance Evaluation

The application's performance can be evaluated using the following parameters:

- Document processing time.
- Semantic retrieval relevance.
- Response generation time.
- Accuracy of generated content.
- Quality of email and letter formatting.
- Reliability of draft history storage.

Actual performance measurements can be included after testing the application with a suitable set of documents and user queries.

6 Conclusion & Future Scope

6.1 Conclusion:

WriteWise AI is an AI-powered email and formal letter writing assistant developed using Retrieval-Augmented Generation technology.

The project combines Python, Streamlit, LangChain, ChromaDB, Sentence Transformers, and Large Language Models to provide a user-friendly platform for generating professional written communication.

By integrating semantic search with context-based generation, the application retrieves relevant information from uploaded reference documents and uses it to generate personalized emails and formal letters.

The application also provides useful features such as document management, editable drafts, grammar refinement, content regeneration, downloadable outputs, and local writing history.

Overall, WriteWise AI demonstrates how Retrieval-Augmented Generation can be applied to improve automated writing assistance by combining external knowledge retrieval with the generative capabilities of Large Language Models.

6.2 Future Scope

The project can be further enhanced by implementing the following features:

1. Multilingual Support: Extend the application to support more regional and international languages.
2. Multiple Export Formats: Provide options to download generated content in PDF and DOCX formats.
3. Cloud-Based Storage: Integrate cloud storage to access documents and writing history across multiple devices.
4. User Authentication: Implement secure user accounts and personalized document access.
5. Advanced Retrieval: Improve retrieval quality using hybrid search, reranking, and more advanced embedding models.
6. Voice-Based Input: Allow users to provide writing requirements through speech.
7. Additional Writing Categories: Extend support to reports, resumes, notices, applications, and other professional documents.
8. Feedback Mechanism: Allow users to rate generated drafts and use feedback to improve content quality.
9. Multimodal RAG: Support extraction and processing of images, tables, and charts from uploaded documents.

These enhancements can make WriteWise AI a more flexible, intelligent, and comprehensive writing assistant.